UNIVERSITY OF GHANA

DEPARTMENT OF COMPUTER SCIENCE

CSCD602 — ADVANCED SOFTWARE ENGINEERING

INDIVIDUAL PROJECT-BASED EXAMINATION

PROJECT DOCUMENTATION

ICCTECH

A Web-Based IT Service Management and Helpdesk System

Item	Details
Student Name	Clement Asamoah
Student ID	22424193
Academic Year	2025/2026
Examination Duration	48 Hours
Live Application	http://45.79.223.146:8080/index.php
Source Repository	https://github.com/Clemzy123/ICCTECH

PROJECT DOCUMENTATION

Contents

Dedicated companion files: SRS.pdf, Testing_Report.pdf, Technical_Debt_Plan.pdf and User_Manual.pdf.

1. Introduction, Stakeholders and Requirements Summary

Problem. Fragmented IT support through calls, email, messaging, spreadsheets and verbal reports causes lost or duplicated requests, unclear ownership, inconsistent priority/status, incomplete history and weak service visibility.

Aim. ICCTECH provides a centralised browser-based ITSM/helpdesk workflow for recording, assigning, tracking, communicating on and resolving support requests while demonstrating the required Advanced Software Engineering lifecycle.

Stakeholders and roles

- **End users:** authenticate, submit and track permitted tickets, reply to support staff and use published knowledge.
- **Support analysts:** triage, categorise, prioritise, assign, investigate, communicate, update status and resolve tickets.
- **Administrators:** manage users, analysts, roles, permissions, teams, settings and permitted operational information.
- **Managers/knowledge/asset stakeholders:** use service information, reusable knowledge and equipment records; the examiner validates traceable engineering evidence.

Requirements and scope

The formal baseline contains 16 functional requirements and 12 non-functional requirements. Must-Have scope covers authentication, RBAC, the complete ticket lifecycle and administration; knowledge and assets are Should-Have; basic audit/operational information is Could-Have. Quality requirements cover usability, representative performance, security, persistent MySQL storage, availability, compatibility, Docker deployability, maintainability and responsive use. Full FR/NFR definitions, use cases, acceptance criteria and traceability are in SRS.pdf.

Core workflow: Submit ticket → triage → categorise/prioritise → assign → investigate/communicate → resolve → close → reopen when further work is required.

2. Software Effort Estimation

Effort is estimated with a bottom-up three-point (PERT) model. For each activity, optimistic (O), most-likely (M) and pessimistic (P) hours are recorded and the expected value is computed as (O + 4M + P) / 6. The estimate covers analysis, design, implementation, deployment, testing and documentation within the 48-hour examination.

2.1 Effort Calculation

Table 2.1. Bottom-up three-point (PERT) effort calculation by project activity

Project Activity	O	M	P	Expected Hours
Define problem, users and requirements	1.0	2.5	4.0	2.5
Analyse application requirements and technical feasibility	2.0	3.0	4.0	3.0
Review/design architecture, database and workflow	2.0	3.0	4.0	3.0
Implement/validate core ticket workflow	4.0	6.0	8.0	6.0
Implement/validate authentication and role access	2.0	3.0	4.0	3.0
Configure/validate knowledge and asset functions	1.0	2.5	4.0	2.5
Configure Docker and deploy to Linode	2.0	4.0	6.0	4.0
Execute functional and security tests	3.0	5.0	7.0	5.0
Correct defects and perform regression testing	2.0	3.0	4.0	3.0
Prepare project/deployment documentation	2.0	3.0	4.0	3.0
Final verification and submission checks	0.5	1.0	1.5	1.0

Estimated effort

Total planned work = 36 person-hours. A four-hour contingency reserve is added for unexpected technical issues, producing an estimated total of 40 person-hours.

2.2 Estimated Development Duration

The examination provides 48 elapsed calendar hours. With one student developer, 40 person-hours represent approximately 40 active working hours. The remaining eight hours provide allowance for rest, meals, short interruptions and unavoidable non-project activity.

Table 2.2. Planned distribution of the 48-hour examination period

Calendar Period	Main Activity	Active Hours	Break / Other
Hours 1–6	Problem definition, stakeholders, requirements and scope	5	1
Hours 7–12	Technical analysis, architecture and design	5	1
Hours 13–24	Core workflow and access-control work	10	2
Hours 25–30	Docker/Linode deployment	4	2
Hours 31–38	Testing and defect correction	7	1
Hours 39–44	Documentation and evidence	5	1
Hours 45–48	Contingency, verification and submission	4	0

2.3 Assumptions

- One student performs all project activities.
- The Linode server and internet connection remain available.
- PHP 8.4, Apache, MySQL 8.0, Docker and Docker Compose can be used in the target environment.
- The core ticket workflow is the main implementation and validation focus.
- Third-party integrations that require external credentials are outside the critical path.
- Testing and documentation are included in the person-hour estimate.
- The contingency reserve is used only for unexpected technical problems.

2.4 Constraints

- Fixed 48-hour examination duration.
- Single developer.
- Large codebase with functionality outside the examination scope.
- Limited time for exhaustive testing of every module.
- Deployment depends on cloud, network, container and database availability.
- Documentation, testing and implementation compete for the same limited time.

3. System Analysis

3.1 Analysis of the Existing Problem

The existing support-management problem is characterised by fragmented reporting channels and weak traceability. A phone call or instant message can communicate a problem quickly, but it does not necessarily create a persistent operational record. This makes status tracking, ownership, prioritisation, historical analysis and audit difficult.

Table 3.1. Observed problems, effects and the required system response

Observed Problem	Effect	Required System Response
Requests arrive through informal channels	Requests may be forgotten or duplicated	Create a central ticket record with a unique reference.
Ownership is unclear	Users do not know who is responsible	Support assignment and visible ticket ownership.
No consistent priority	Urgent issues may compete with routine requests	Configurable priority and triage.
Updates are scattered	Communication history is incomplete	Record messages/notes against the ticket.
Status is not visible	Users repeatedly ask for progress	Ticket status and self-service visibility.
Solutions are not reused	Analysts repeatedly solve the same problem	Knowledge-base capability.
Device context is separated	Troubleshooting lacks asset history	Asset records and user/asset associations.
Limited accountability	Difficult to review what changed	Ticket audit and system logging.

3.2 Proposed System Behaviour

ICCTECH converts each support request into a persistent ticket and guides it through a controlled lifecycle. The system separates requester-facing activities from analyst and administrator functions, while using authentication and role/capability checks to protect administrative actions.

3.3 Core Business Process

Submitted → Open / triage → Categorise + Prioritise → Assign → In Progress → On Hold / Awaiting Response where necessary → In Progress → Resolve → Close → Reopen if further work is required.

The source database defines configurable ticket statuses rather than hard-coding business meaning into a single fixed workflow. Default statuses include Open, In Progress, On Hold, Awaiting Response and Closed. On Hold and Awaiting Response can pause the service-level agreement clock by default.

3.4 Data Analysis

The core data model revolves around tickets and their relationships to users, analysts, statuses, priorities and departments. Ticket notes and ticket audit records preserve operational history. Asset and user-asset records provide device context, while knowledge articles provide reusable support information. RBAC tables link analysts to roles and roles to granular capabilities.

3.5 Role and Permission Analysis

The analyst authentication flow includes password verification, failed-login tracking, account lockout and optional TOTP multi-factor authentication. Authorisation is implemented beyond interface visibility: server-side role and capability checks are used to determine whether an analyst can access protected modules or administrative functions. This supports the requirement that access control must be enforced according to role rather than simply hiding interface elements.

3.6 Feasibility Analysis

Table 3.2. Feasibility assessment across five dimensions

Dimension	Assessment
Technical feasibility	High. The required stack is available in the repository: PHP 8.4/Apache, MySQL 8.0, PDO, Docker and Docker Compose.
Operational feasibility	High for the core workflow. End users, analysts and administrators have clear roles and a browser-based interface.
Schedule feasibility	Feasible only with strict scope control. The 48-hour limit requires prioritising the core ticket lifecycle and evidence.
Deployment feasibility	High. The application is containerised and already deployed to a Linode Linux server.
Security feasibility	Reasonable for the examination scope, but production hardening such as HTTPS, secret rotation and infrastructure controls must be treated as ongoing work.

4. System Design — Architecture

The design selects artefacts that best communicate the application rather than producing every possible UML diagram. The most useful views for ICCTECH are the deployment/system architecture, use cases, ticket activity flow, core database relationships and a representative sequence for ticket creation and update.

The deployed design separates browser access, the Apache/PHP application service and MySQL database under Docker Compose on Linode. PDO provides application-to-database access and persistent volumes retain database data, attachments and keys.

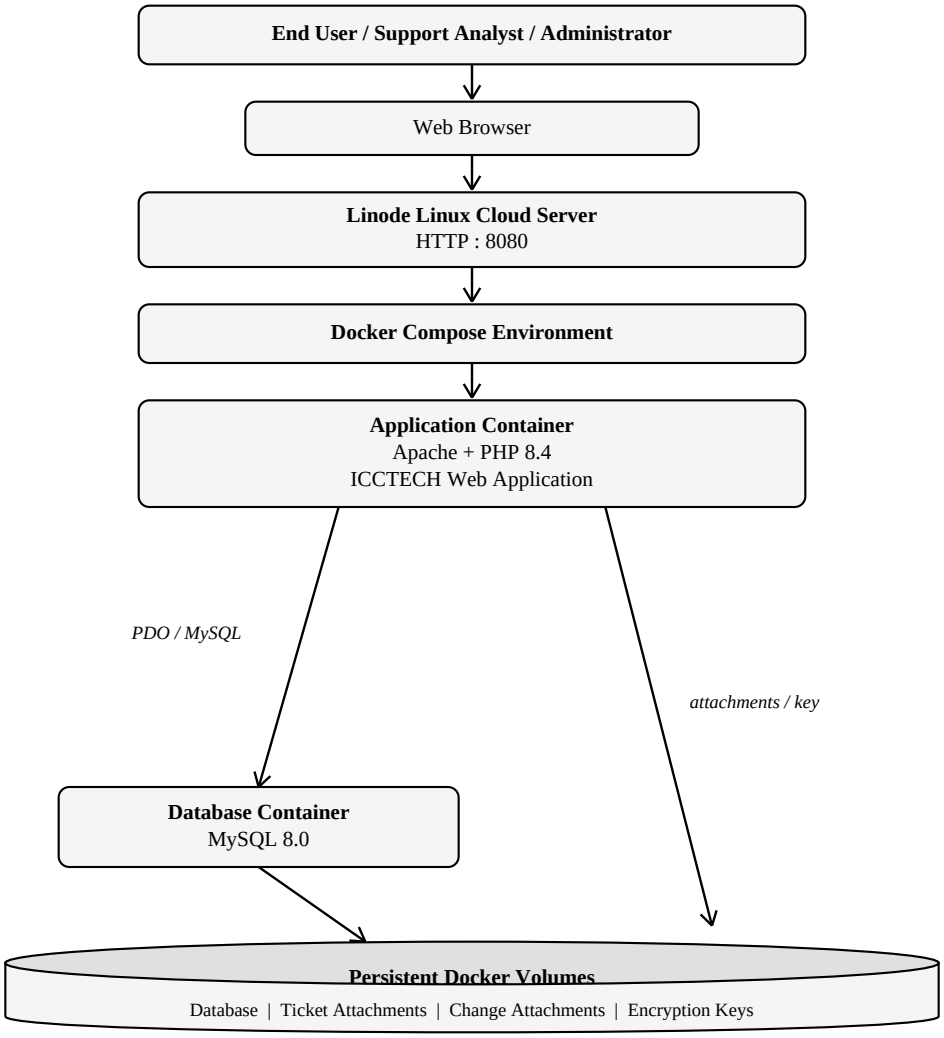

Figure 4.1. ICCTECH containerised deployment architecture

Users access the application through a web browser. The Linode host exposes port 8080, which Docker maps to Apache port 80 inside the application container.

4. System Design — Use Cases

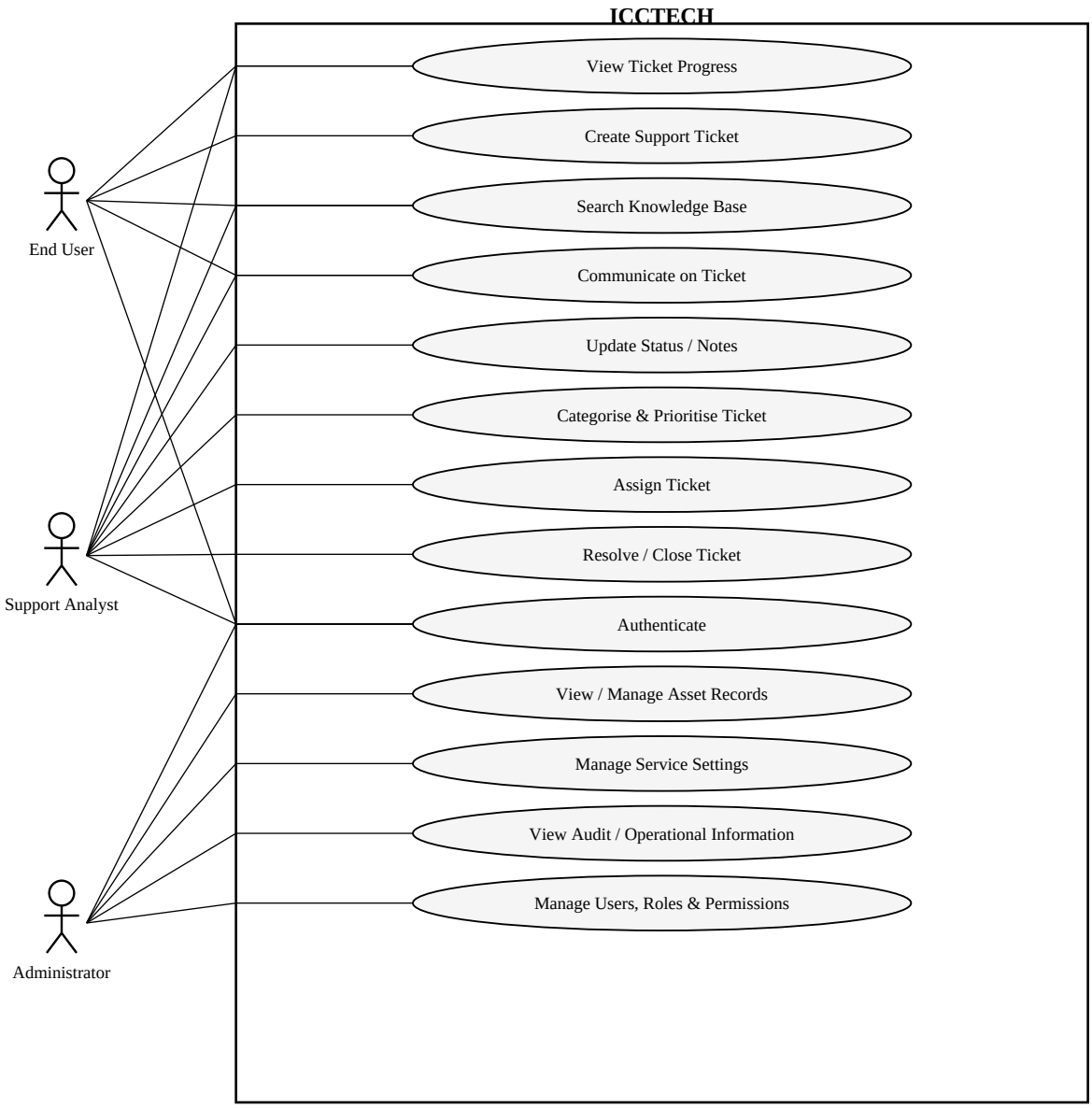

Figure 4.2. Core ICCTECH use cases by user class

The use-case design separates end-user, analyst and administrator responsibilities while keeping authentication as the protected entry point. End users focus on reporting and monitoring support requests. Analysts manage the operational ticket lifecycle. Administrators control users, roles, service settings and selected audit information. Detailed behavioural specifications are maintained in SRS.pdf.

4. System Design — Ticket Activity

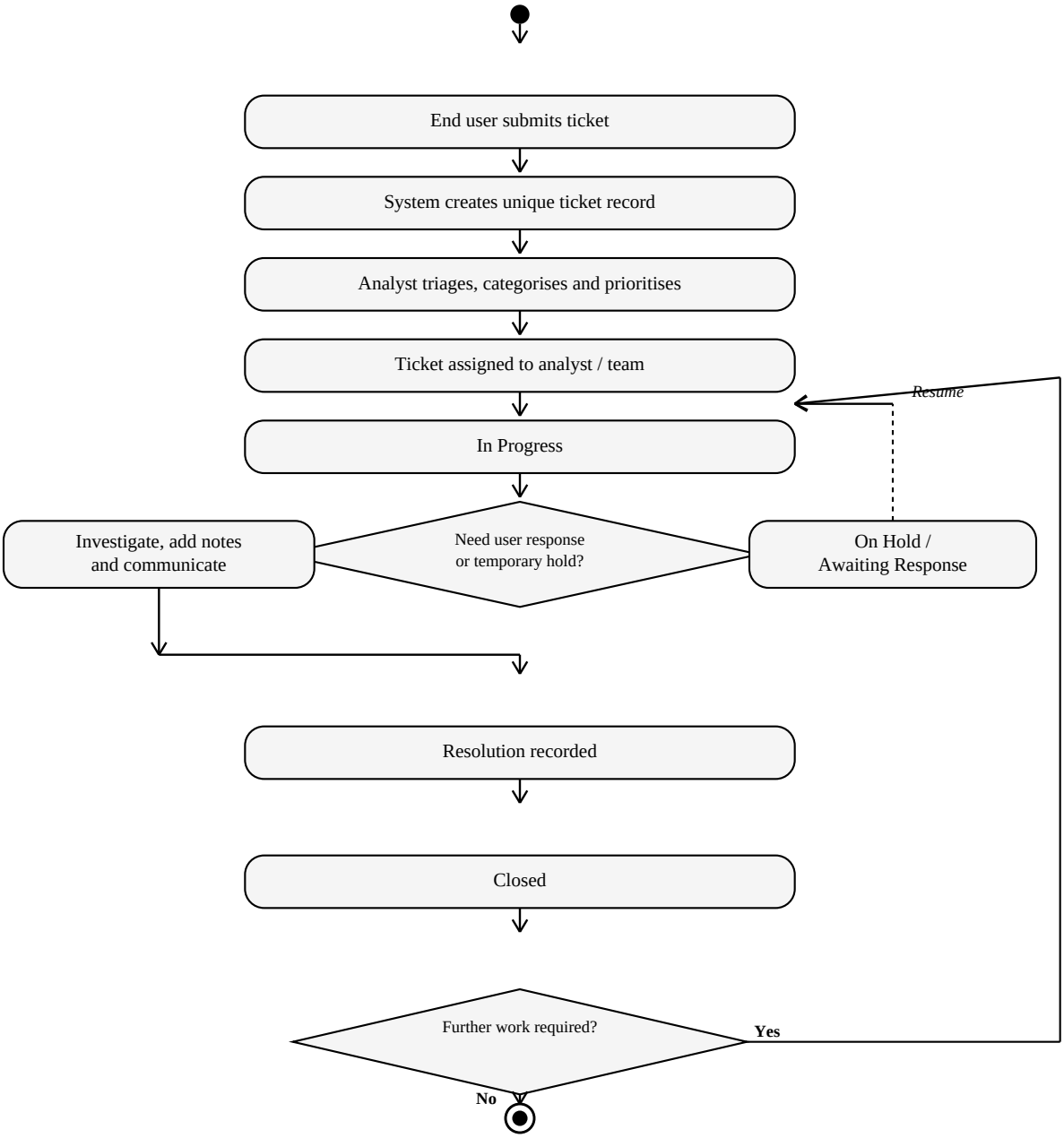

Figure 4.3. Core ticket lifecycle activity flow

The activity flow is the primary service-desk process and includes the hold/awaiting-response decision, resolution, closure and controlled reopen path.

4. System Design — Core Data Model

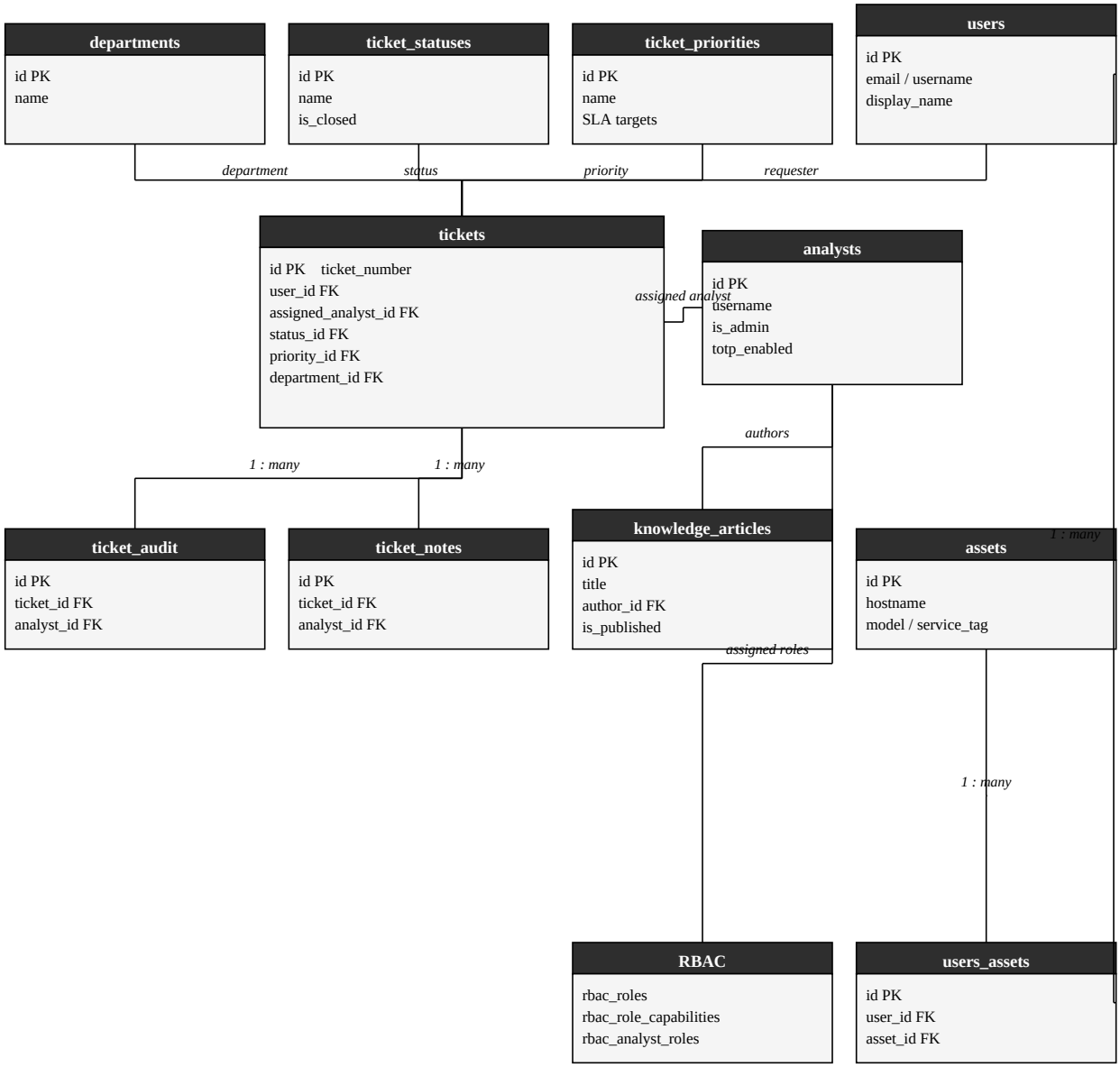

Figure 4.4. Simplified ER view of examination-scope entities

Core entities include users, analysts, tickets, statuses, priorities, departments, notes, audit records, assets, user-asset links, knowledge articles and RBAC role/capability tables. Foreign-key relationships retain requester, ownership, history and access-control context. The full database schema is substantially larger than the examination scope; the diagram focuses on the entities required to explain the core helpdesk workflow.

4. System Design — Representative Sequence

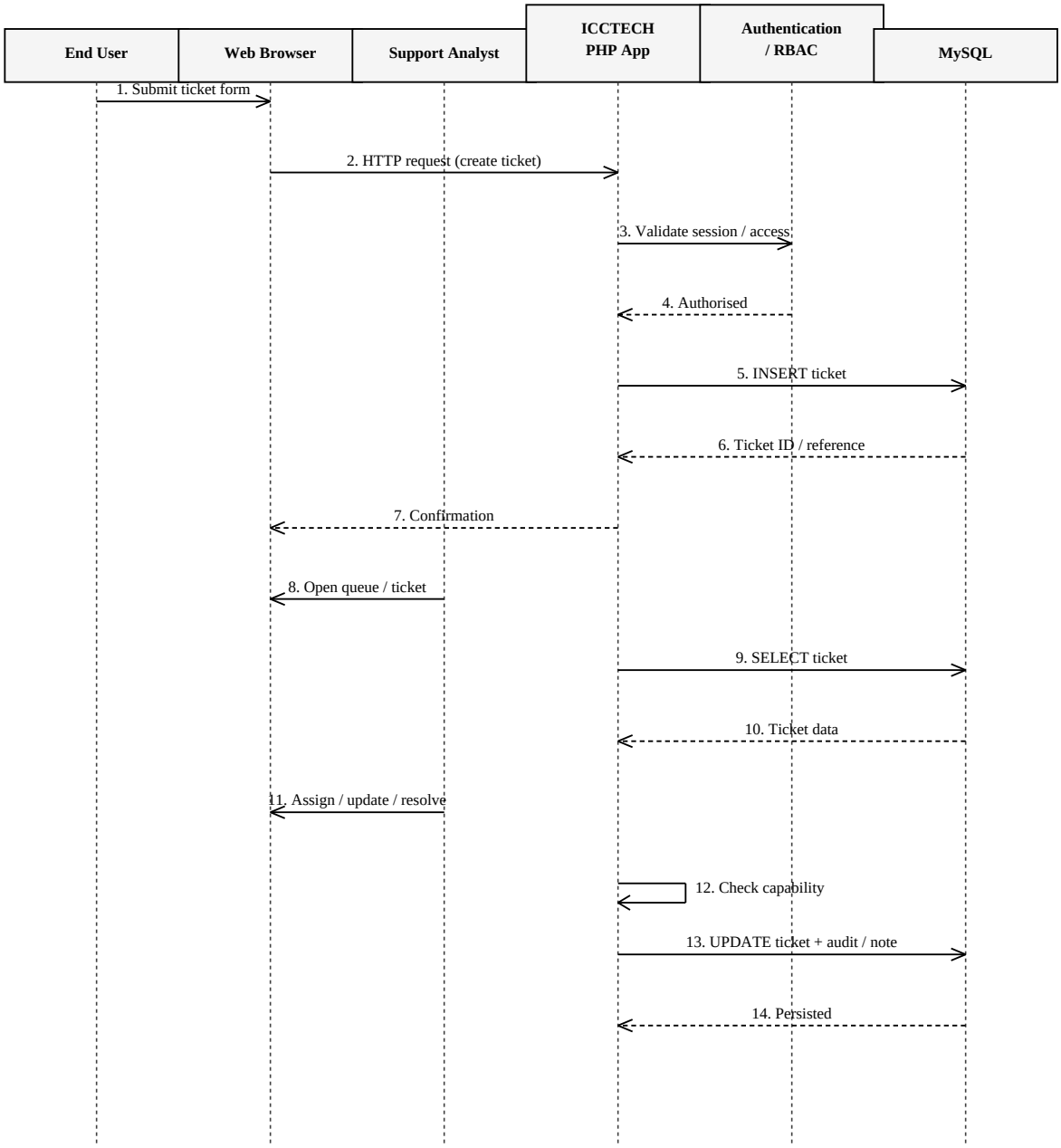

Figure 4.5. Representative sequence for ticket creation and analyst update

The sequence shows validated ticket creation and subsequent analyst update: authenticated requests are authorised, persisted in MySQL and accompanied by audit/note history. Server-side capability checks protect privileged updates.

5. Implementation

5.1 Technology Stack

Table 5.1. Implementation evidence for each technology component

Component	Implementation Evidence / Purpose
PHP 8.4 + Apache	The Dockerfile is based on php:8.4-apache and enables the required PHP extensions.
MySQL 8.0	docker-compose.yml defines a separate MySQL 8.0 service and imports database/freeitsm.sql on initial creation.
PDO / pdo_mysql	Application database communication uses PDO and the MySQL driver.
Docker / Docker Compose	Application and database are containerised and orchestrated as separate services.
HTML / CSS / JavaScript	Provides browser-based user and administrative interfaces.
Git / GitHub	Maintains the source repository and project history.
Linode Linux server	Hosts the live deployment on port 8080.

5.2 Application Structure

The codebase is organised into web modules, API endpoints, shared includes/services, database scripts, Docker configuration, documentation and tests. Relevant paths for the examination scope include *auth/*, *api/auth/*, *api/tickets/*, *asset-management/*, *knowledge/*, *includes/*, *database/*, *docker/* and *tests/*.

Key project files
Dockerfile | docker-compose.yml | auth/login.php | includes/rbac.php | api/tickets/create_ticket.php | api/tickets/assign_ticket.php | api/tickets/get_ticket_audit.php | database/freeitsm.sql

5.3 Authentication Implementation

The analyst login implementation in *auth/login.php* retrieves analyst authentication state, verifies local passwords with *password_verify()*, tracks failed login attempts and supports account lockout. The same flow also contains optional TOTP multi-factor authentication and trusted-device handling. Password reset code uses *password_hash()* with PHP's default password algorithm. The source also contains optional external authentication integrations, but these are not required for the core examination workflow.

5.4 Authorisation and RBAC

Role-based access is implemented through analyst roles and granular capabilities. The database contains *rbac_roles*, *rbac_role_capabilities* and *rbac_analyst_roles*. Shared RBAC code in *includes/rbac.php* and related settings code performs capability checks for protected functions. This is important because security is enforced server-side rather than relying only on whether a menu item is visible.

5.5 Ticket Management

The ticket subsystem exposes endpoints for ticket creation, assignment, status/priority retrieval, user-ticket viewing, notes, audit history, restore/reopen operations, deletion/trash handling and dashboard data. The central *tickets* table links the request to a user, assigned analyst, status, priority and department. Related *ticket_notes* and *ticket_audit* records preserve communication and change history.

5.6 Knowledge and Asset Functions

Knowledge articles are persisted in *knowledge_articles* with author, publication, review and version information. Asset records include device identity and lifecycle attributes such as hostname, manufacturer, model, operating system, service tag, status and location. *users_assets* links assets to end users and records who made the assignment.

5.7 Database Persistence

MySQL 8.0 provides persistent storage. The initial schema is stored in *database/freeitsm.sql* and Docker Compose mounts the database directory to a named volume. This ensures that operational records are not stored only inside a disposable container layer.

5.8 Validation, Error Handling and Security Controls

- Password hashing and verification using PHP password functions.
- Failed-login counting and configurable lockout logic for analysts.
- Optional TOTP MFA support.
- Role and capability checks for protected analyst functions.
- Prepared PDO statements are used throughout key database operations.
- Persistent encryption-key storage outside the web document root in the Docker image design.
- Application separation from the database through the Docker internal service network.
- Input validation and server-side checks are used by application/API endpoints where relevant.

5.9 Requirement-to-Source Traceability

Table 5.2. Examination evidence for persistence, deployability, testing and source control

Requirement / Area	Source Evidence	Examination Contribution Demonstrated
NFR-06, NFR-09, NFR-11 — Persistence & deployability	Dockerfile; docker-compose.yml; database/freeitsm.sql; named volumes	Configured and deployed the PHP/MySQL application on Linode using Docker Compose, then verified database-backed persistence after controlled service restart.
Testing and deployment evidence	tests/; live Linode deployment; GitHub repository	Executed the examination-specific test cycle, documented results, and maintained the deployable source repository for examiner verification.

Traceability note

This table records what was demonstrated and validated for the examination. It does not represent every upstream module in the wider repository as newly authored during the 48-hour period.

5.10 Reuse and Third-Party Components

Academic-integrity statement

The repository contains substantial upstream FreeITSM functionality and third-party components. The examination documentation should distinguish the student's project-specific analysis, configuration, implementation, validation, deployment and documentation work from inherited platform functionality. The upstream project and relevant libraries/frameworks should be acknowledged in the References section rather than representing the entire codebase as newly authored during the 48-hour examination.

5.11 Implementation Outcome

The implemented and deployed project provides the functional foundation required to demonstrate the prioritised ITSM workflow in a live environment. The requirement-to-source traceability above, together with screenshots, final test results and repository history, allows the examiner to connect documented requirements to working functionality while keeping project-specific examination work distinct from inherited platform components.

6. Testing and Technical Debt Summary

Testing

The final examination cycle executed 17 formal tests against the deployed Linode/Docker environment. All 17 passed; none failed or were blocked, and no final-cycle defect remained open. Coverage includes valid/invalid authentication, RBAC, ticket creation through reopen, administration, knowledge, assets, persistence after service restart, live deployment, representative page-response performance and responsive-interface usability. Full procedures, expected/actual results and traceability are in Testing_Report.pdf.

Technical debt

- **TD-01 Critical:** externalise and rotate development/default credentials.
- **TD-02 Critical:** replace HTTP-only public access with HTTPS/TLS.
- **TD-03 High:** remove or strictly restrict the published MySQL host port.
- **TD-04 High:** expand requirements-mapped automated regression and add CI.
- **TD-05 High:** implement independent off-host backup and tested restore.
- **TD-06 Medium:** add production monitoring and automated alerts.
- **TD-07 Medium:** single Linode host remains a resilience/scalability limitation.
- **TD-08 Medium:** refactor legacy/upstream internal naming incrementally.
- **TD-09 Medium:** freeze/tag the final release to prevent documentation/evidence drift.

The complete Debt → Cause → Impact → Priority → Proposed Resolution records and repayment roadmap are in Technical_Debt_Plan.pdf.

7. Deployment and Accessibility

The application is deployed on a Linode Linux host with Docker Compose. Host port 8080 maps to Apache in the PHP application container; MySQL 8.0 runs as a separate service and named volumes persist database/application data. Deployment verification covers service startup, database connectivity, authentication, ticket creation and retrieval, restart persistence and live browser reachability. Production hardening priorities are HTTPS, externalised secrets, restricted database exposure, off-host backups and monitoring.

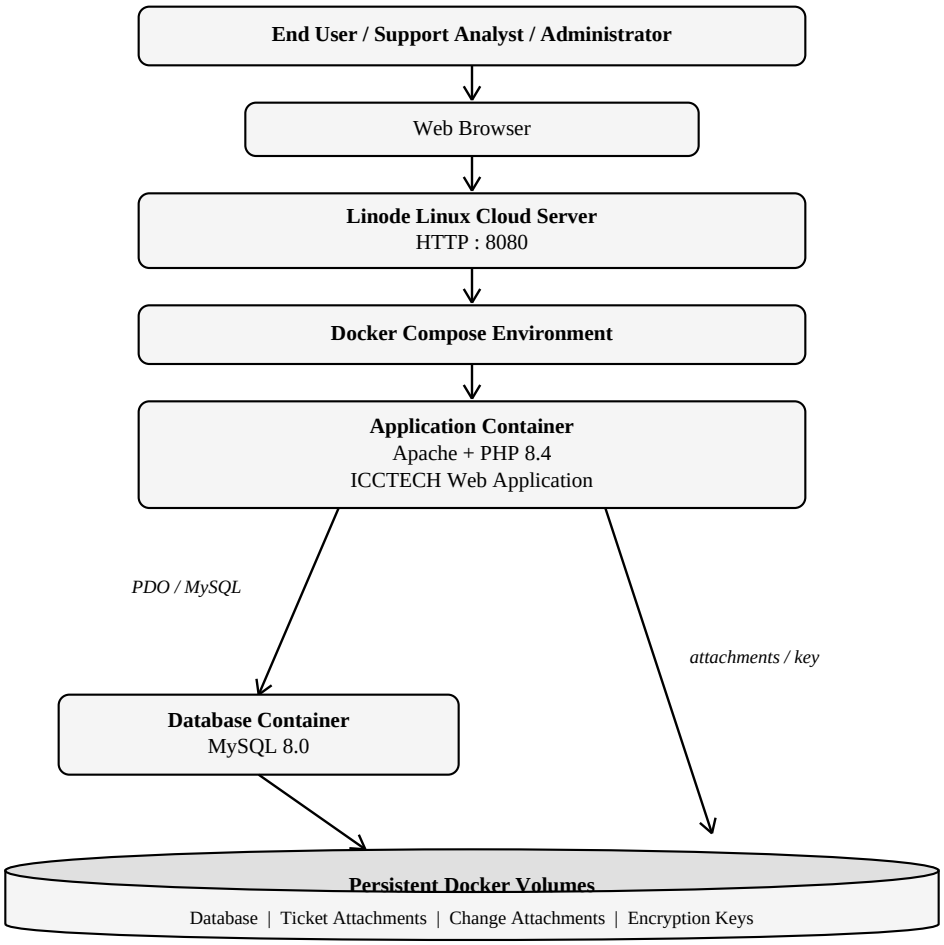

Figure 7.1. Deployment architecture

8. Maintenance, Future Evolution, Limitations and Conclusion

Maintenance. Corrective work addresses verified defects; adaptive maintenance handles platform/dependency/environment changes; perfective work improves usability, workflow and reporting; preventive maintenance covers updates, backups, secret rotation, access review and health/log monitoring. Releases should use identified Git revisions, regression testing, rollback planning and updated documentation.

Future evolution. Near-term priorities are HTTPS, CI/regression automation, off-host backup/restore verification, monitoring and refined dashboards. Later evolution can add governed identity/integration features, SLA/escalation automation, richer knowledge/asset links and higher-availability architecture when justified by scale.

Limitations. The 48-hour examination restricts long-term UAT, exhaustive cross-browser/device testing, penetration testing, large-scale load testing, disaster-recovery exercises and broad enterprise customisation. The IP-based HTTP deployment is suitable for demonstration rather than hardened production. Substantial FreeITSM/upstream capability is reused and acknowledged; repository size is not claimed as work authored entirely during the examination.

Conclusion. ICCTECH demonstrates the complete software-engineering lifecycle: scoped requirements, quantified effort, analysis/design, implementation, deployment, formal verification, technical-debt management, documentation, maintenance and evolution. The prioritised workflow is demonstrable online and all 17 formal acceptance tests passed.

9. References and Acknowledgements

- ICCTECH repository: https://github.com/Clemzy123/ICCTECH and live deployment: http://45.79.223.146:8080/index.php.
- PHP, Apache HTTP Server, MySQL 8.0, Docker/Docker Compose and Linode/Akamai documentation.
- Upstream FreeITSM platform and third-party libraries used by the inherited codebase, acknowledged as reused rather than newly authored during the examination.